Algorithmic Improvement Proposals for Vamana ANN

A Rigorous, Implementation-Oriented Exploration (Consolidated)

Prepared for the GraphANN repository

April 2026

Abstract

This document proposes ten high-impact improvements for a baseline Vamana-style approximate nearest neighbor (ANN) implementation. Proposals 1–5 target the algorithm itself: graph construction quality, candidate generation, parameter adaptivity, search robustness, and distance-computation overuse. Proposals 6–10 target the implementation: the per-query hot loop, vectorized distance, graph memory layout, build-time concurrency, and build-quality diagnostics. For each proposal we give (i) concrete code-level changes, (ii) expected empirical benefits, and (iii) mathematical framing to justify the change under a fixed memory budget at search time. The proposals are prioritized for practical adoption in the C++ `graphann` baseline (`greedy_search`, `robust_prune`, bounded degree R, parallel insertion with per-node mutexes).

1 Baseline Summary and Optimization Goal

Let $P = \{x_i\}_{i=1}^n \subset \mathbb{R}^d$ denote the dataset and let $G = (V, E)$ be a directed graph over points with bounded out-degree. Query-time ANN search uses best-first greedy graph traversal with candidate budget L, returning top-K points by distance.

The practical optimization objective is a constrained multi-objective tradeoff:

$$\min_{G,\ \text{search policy}} \big(\mathbb{E}[\text{latency}],\ \mathbb{E}[\text{distance computations}],\ \text{build time}\big) \quad \text{s.t.} \quad \text{Recall@}K \geq \rho,\ \deg^+(v) \leq R,\ \text{Mem} \leq M. \tag{1}$$

Vamana-style design specifically tries to construct a sparse graph that remains highly navigable under greedy traversal. The ten improvements below target the same constrained objective through better graph quality, better candidate generation, lower per-query compute, and better memory/cache behavior.

Baseline touch points (from `src/vamana_index.cpp`). `greedy_search` uses a `std::set<Candidate>` for the bounded frontier, a `std::set<uint32_t>` to track expanded nodes, and a `std::vector<bool> visited(npts_, false)` allocated per query. `robust_prune` implements the α-RNG rule. `build` picks a random start node, inserts points in a random order under OpenMP with per-node `std::mutex`, and uses a single γR overflow threshold. The distance routine is a scalar `float` loop in `src/distance.cpp`.

2 Improvement 1: Two-Pass Vamana with Medoid Start and α-Scheduling

2.1 What to change

1. Replace the random entry node with a medoid-like center:

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad s = \arg\min_i \|x_i - \mu\|_2^2. \tag{2}$$

2. Initialize the graph with a small over-connected random seed instead of fully empty adjacency.
3. Perform two full insertion passes:
   - Pass 1: strict pruning with $\alpha_1 = 1.0$.
   - Pass 2: relaxed pruning with $\alpha_2 \in [1.1, 1.4]$.
4. Keep degree controls (target R, overflow γR), optionally with pass-specific L values.

2.2 Why it should be better

A random start node increases expected initial search distance and biases early graph shaping; a medoid-like entry improves routing from the first hop. Single-pass construction is path-dependent; two-pass refinement reduces local construction artifacts. $\alpha_1 = 1$ preserves local geometric precision; $\alpha_2 > 1$ reintroduces strategic long-range links that reduce diameter and hop count.

2.3 Mathematical perspective

Define expected hop count to an acceptable neighborhood of true nearest neighbors as $H(q; G, s)$. Under fixed degree budget we seek

$$\min_G \ \mathbb{E}_q[H(q; G, s)] \quad \text{s.t.} \quad \deg^+(v) \le R. \tag{3}$$

Interpret two-pass Vamana as continuation in pruning aggressiveness:

$$G^{(1)} = \mathcal{P}_{\alpha=1}(G_0), \qquad G^{(2)} = \mathcal{P}_{\alpha=\alpha_2}(G^{(1)}), \quad \alpha_2 > 1. \tag{4}$$

Pass 1 sharpens local structure; Pass 2 broadens navigability. Empirically this decreases graph diameter and high-percentile query hops while keeping degree bounded.

2.4 Cost impact

Build time typically increases by 1.7×–2.0×, but query-time L for a target recall can often be reduced.

3 Improvement 2: Candidate Set for Prune from Visited Set, Not Only Final Frontier

3.1 What to change

During insertion of point p, greedy search should expose two sets:

- Final bounded frontier $S_L(p)$ (current behavior).

- Full visited/expanded set V_p (new tracking required).

Build prune candidates as

$$C_p = V_p \cup N_{\text{out}}(p) \tag{5}$$

instead of only using frontier elements. Run robust prune on C_p to select up to R neighbors.

3.2 Why it should be better

The final frontier is a compressed, lossy summary of traversal. Many useful bridge nodes are visited but not retained in the top-L frontier. A richer candidate support improves odds of selecting edges that reduce local-minima traps and improve global routing.

3.3 Mathematical perspective

Let $F(A)$ denote the utility of selected neighbors under the proximity+diversity constraints induced by robust pruning. The current optimization is over a restricted domain:

$$\max_{A \subseteq S_L(p),\ |A| \le R} F(A). \tag{6}$$

The improved optimization uses the larger domain:

$$\max_{A \subseteq (V_p \cup N_{\text{out}}(p)),\ |A| \le R} F(A). \tag{7}$$

Since $S_L(p) \subseteq V_p \cup N_{\text{out}}(p)$, the optimum cannot decrease.

3.4 Cost impact

Prune stage costs increase because the candidate list is larger; however, better graph quality reduces required query beam width for the same recall.

4 Improvement 3: Locally Adaptive α_i and Degree Budget R_i

4.1 What to change

1. Estimate local intrinsic dimensionality (LID) per point i from local distance statistics:

$$\widehat{m}_i = \left(-\frac{1}{k-1} \sum_{j=1}^{k-1} \log \frac{r_j(i)}{r_k(i)} \right)^{-1}, \tag{8}$$

   where $r_j(i)$ is the j-th nearest local sample distance.

2. Set node-specific pruning aggressiveness and degree:

$$\alpha_i = \text{clip}\left(1 + \frac{c_\alpha}{\sqrt{\widehat{m}_i}},\ \alpha_{\min},\ \alpha_{\max}\right), \qquad R_i = \text{clip}\big(R_0 + c_R\, \widehat{m}_i,\ R_{\min},\ R_{\max}\big). \tag{9}$$

3. Replace global prune parameters with per-node values in `robust_prune` and overflow logic (γR_i threshold per node).

4. Keep total memory bounded by enforcing

$$\sum_{i=1}^{n} R_i \le n\, R_{\text{budget}}. \tag{10}$$

4.2 Why it should be better

Real datasets are locally heterogeneous. Uniform α and R over-prune hard regions and over-allocate easy ones. Adaptive budgets allocate edges where marginal recall gain per edge is larger.

4.3 Mathematical perspective

Global policy:

$$\min_G \sum_i L_i(G; \alpha, R). \tag{11}$$

Adaptive policy:

$$\min_G \sum_i L_i(G; \alpha_i, R_i) \quad \text{s.t.} \quad \sum_i R_i \leq n\, R_{\text{budget}}. \tag{12}$$

This is a constrained resource allocation problem; LID is a proxy for local geometric complexity and expected branching needs.

4.4 Cost impact

Small overhead for LID estimation and metadata storage; potentially strong recall–latency gains at fixed average memory.

5 Improvement 4: Multi-Entry Search and Query-Adaptive Beam Width

5.1 What to change

1. Replace single start node with M entry points (medoid + diversified pivots/centroids).
2. Seed the search frontier with all entries (shared best-first traversal).
3. Use an adaptive beam width: start with small L_0; increase only when confidence is low or progress stalls.
4. Add a confidence-based stopping criterion using candidate lower bounds and the current k-th best distance.

5.2 Why it should be better

Tail queries far from a single entry basin are brittle. Multiple entries reduce local-minima risk. Adaptive L avoids over-computing on easy queries and spends work only on hard ones.

5.3 Mathematical perspective

If one entry fails with probability p_f and starts are weakly correlated, multi-entry failure behaves as

$$p_f^{(M)} \approx p_f^M. \tag{13}$$

For stopping, let d_k be the current exact k-th best and ℓ_{next} a lower bound on unexpanded candidates. Stop when

$$\ell_{\text{next}} \geq d_k(1 - \varepsilon), \tag{14}$$

which bounds the expected gain from additional expansions.

5.4 Cost impact

Low metadata overhead for extra entries; often large reduction in high-percentile latency at constant recall.

6 Improvement 5: Distance Cascade (Approximate Screening + Exact Rerank)

6.1 What to change

1. Store a compact approximate representation per point (scalar-quantized int8, or Product Quantization codes).
2. In greedy expansion and pruning pairwise checks, compute the cheap approximate distance $\tilde{d}$ first.
3. Compute exact distance d only for the top-B approximate candidates or ambiguous margin cases.
4. Keep final top-K reranking exact to preserve output quality.

6.2 Why it should be better

Distance computation dominates CPU cost in both build and query in graph-based ANN. Most candidates are clearly inferior and can be rejected with cheap approximate scores. Under a fixed memory ceiling, compressed codes also free budget that can be redirected into higher R or larger caches.

6.3 Mathematical perspective

Assume bounded approximation error:

$$|d(x, y) - \tilde{d}(x, y)| \leq \delta. \tag{15}$$

If candidates a, b satisfy

$$\tilde{d}(a) + \delta < \tilde{d}(b) - \delta, \tag{16}$$

then exact ordering is guaranteed ($d(a) < d(b)$), so one exact evaluation can be skipped safely. This yields

$$\mathbb{E}[C_{\text{exact}}] \approx \rho\, \mathbb{E}[C_{\text{total}}], \qquad \rho \ll 1, \tag{17}$$

while maintaining quality through exact final reranking.

6.4 Cost impact

Additional memory for the codes and one-time calibration effort, but often the largest raw speedup in CPU-bound regimes.

7 Improvement 6: Hot-Loop Rewrite of `greedy_search`

7.1 What to change

The baseline `greedy_search` has three serious per-query inefficiencies that do not affect correctness but cost a large constant factor:

1. **Replace `std::set<Candidate>` with a sorted flat array of size L.** Every frontier insert/erase currently triggers a red-black tree allocation. A sorted `std::array<Candidate, L_MAX>` (or `std::vector<Candidate>` reserved to L) supports insertion via `std::lower_bound` plus a shift: cache-friendly, branchless in the common case, zero heap allocation.

2. **Replace the "scan for first un-expanded" loop with a cursor.** The baseline scans `candidate_set` from the beginning on every iteration (lines 59–65 of `vamana_index.cpp`), giving $O(L^2)$ per query in the worst case. With a sorted array, maintain an integer cursor pointing at the first unexpanded entry; advance it on pop. When a new element is inserted at position $p <$ cursor, set cursor $\leftarrow p$. This makes "find next to expand" $O(1)$. The separate `std::set<uint32_t> expanded` is no longer needed.

3. **Replace `std::vector<bool> visited(npts_, false)` with an epoch-based scratch buffer.** The per-query allocation zero-writes $\lceil n/8 \rceil$ bytes before search even starts; for SIFT1M that is ≈ 125 KB written per query. Instead, keep a per-thread persistent `std::vector<uint32_t> visited_epoch(npts_, 0)` and a `uint32_t current_epoch` incremented once per query. A node is visited iff `visited_epoch[id] == current_epoch`. Clearing is $O(1)$ (just bump the counter); only wrap-around requires a real reset.

7.2 Why it should be better

These are pure implementation wins: same algorithm, same graph, same recall, substantially lower constant factor. On a memory-latency-bound workload like graph ANN, removing allocator traffic and reducing L1-cache pollution directly translate to more distance evaluations per second.

7.3 Cost impact

No memory increase at search time (per-thread scratch buffers are $O(n)$ each but independent of L). Build time also benefits because `build` calls `greedy_search` once per point. Expected throughput uplift: roughly 2×–5× QPS at the same recall, before touching any math.

8 Improvement 7: Explicit SIMD Distance with Prefetching

8.1 What to change

The baseline `compute_l2sq` is a scalar `for`-loop relying on `-O3 -march=native` auto-vectorization. This is fragile: GCC does not always emit FMA, handles tails poorly, and gives no prefetch.

1. Provide an AVX2 version (8-wide `float`) using `_mm256_fmadd_ps` for the multiply–accumulate, and an AVX-512 version (16-wide) gated on `__AVX512F__`. Dispatch at compile time or use a function pointer set once at startup.

2. Handle the tail (when `dim` is not a multiple of 8 or 16) with a masked load or a scalar loop over the remainder.

3. Inside the expansion loop of `greedy_search`, prefetch the next neighbor's vector: before computing the distance to `neighbors[i]`, issue `_mm_prefetch(get_vector(neighbors[i+1]), _MM_HINT_T0)`. Graph ANN is bandwidth- and latency-bound on the data array, so this alone is worth ∼20–40% throughput.

8.2 Why it should be better

Distance evaluation is the innermost operation in both build and search. Tightening it affects every other metric. Prefetching hides DRAM latency that the out-of-order core cannot otherwise cover because the next vector's address is data-dependent on the adjacency list.

8.3 Cost impact

Code size increases modestly; no runtime memory overhead. Raw QPS uplift typically 1.5×–2.5× on top of the hot-loop rewrite.

9 Improvement 8: Flat Adjacency Layout (CSR) and Lock Removal at Search Time

9.1 What to change

The baseline stores the graph as `std::vector<std::vector<uint32_t>>`, with a `std::mutex` per node. This has three costs:

- A pointer indirection per neighbor-list access.
- Per-node heap allocation — neighbor lists are scattered in memory.
- A `std::mutex` is ∼40 bytes; on an index with $n = 10^7$ nodes that is ∼400 MB of pure lock overhead.

After build, freeze the graph into a Compressed Sparse Row (CSR) layout:

```
uint32_t* neighbors_flat;  // size = sum of degrees
uint64_t* offsets;         // size = npts + 1; neighbors of i are
                           //   neighbors_flat[offsets[i] .. offsets[i
                               +1])
```

Or, since degree is bounded by R, allocate a fixed `uint32_t[npts][R]` block with a separate `uint8_t degrees[npts]`. Neighbor lookup is then one cache line per node. Drop the `locks_` array entirely at search time — mutexes are only needed during build.

9.2 Why it should be better

Search traverses adjacency lists as its outer loop. A flat layout gives linear memory access within a list, eliminates the pointer chase through the outer `vector`, and makes prefetching predictable. The memory freed by dropping the lock array is significant; under a fixed memory ceiling that budget can be redirected into more neighbors (higher R) or into a larger PQ codebook for Improvement 5.

9.3 Cost impact

Build-time change: emit a CSR array once at the end of `build`. Search-time memory drops (no lock array, no per-list allocator overhead). QPS improves both from cache effects and from the predictability enabling Improvement 7's prefetch.

10 Improvement 9: Build Concurrency — Reduce Hub-Node Contention

10.1 What to change

The baseline protects each node with a `std::mutex` and uses `#pragma omp parallel for schedule(dynamic, 64)`. Late in the build, hub nodes accumulate many backward edges; many threads contend on the same lock, serializing the critical step.

Two mitigations, in order of effort:

1. **Batched back-edges.** Each thread accumulates its backward edges into a thread-local buffer of (target, source) pairs. Every B insertions (or at barriers), the buffer is flushed into the graph, sharded by `target % num_threads`, so each thread only writes to its own shard. Re-prune on overflow is deferred until flush.
2. **Lock-free append into fixed-capacity adjacency.** Because the final degree is bounded by γR, pre-allocate `uint32_t[npts][gamma*R]` with an atomic counter per node. Appending a back-edge is a single `fetch_add` on the counter; overflow triggers pruning under a per-node lock acquired lazily.

10.2 Why it should be better

With uniform per-node locks and a small number of very high-degree nodes, the build's parallel speedup plateaus sharply — often before 16 threads. Reducing contention on hot nodes improves scaling and makes experimentation cycles (two-pass builds, parameter sweeps) noticeably faster.

10.3 Cost impact

No search-time impact; purely a build-time change. The batching variant is a modest code change; the lock-free variant requires care with the prune path.

11 Improvement 10: Degree-Distribution Diagnostics and Occlusion-Aware Pruning

11.1 What to change

1. Log the full out-degree histogram after build, plus reverse-degree (in-degree) histogram. Compute percentiles ($p50, p90, p99$) and the number of points at the γR ceiling.
2. If the distribution is heavy-tailed — a few super-hubs absorbing most back-edges — tighten γ (e.g. from 1.5 to 1.2). The total neighbor-storage cost and the average hop cost both drop.
3. Replace the pure α-RNG check in `robust_prune` with an occlusion-aware variant: among candidates that pass the distance test, prefer those whose direction from `node` is not already covered by a selected neighbor. Concretely, given the unit vector $\hat{u}_c = (x_c - x_{\text{node}})/\|x_c - x_{\text{node}}\|$, reject c if $\max_{n \in A} \langle \hat{u}_c, \hat{u}_n \rangle > \tau$ for some threshold τ (e.g. 0.95).

11.2 Why it should be better

The α-RNG rule is proximity-aware but only weakly direction-aware. Two candidates along nearly the same ray can both pass the distance test and produce redundant edges, wasting part of the

R budget. Occlusion pruning diversifies edge directions at fixed R, which is exactly what greedy search benefits from near local minima.

11.3 Mathematical perspective

With fixed degree budget R, the expected reduction in distance per greedy hop is bounded by the *angular coverage* of outgoing edges. If outgoing directions are drawn from a distribution with angular dispersion σ, the probability that none of R neighbors makes progress toward the query is roughly $(1 - g(\sigma))^R$ for some monotone g. Maximizing $g(\sigma)$ (edge direction diversity) minimizes the stall probability per hop.

11.4 Cost impact

Diagnostics: negligible. Occlusion pruning: one extra dot product per candidate-already-selected pair, i.e. constant factor on the existing prune loop. Search behavior unchanged except through the improved graph.

12 Implementation Priority and Experimental Plan

12.1 Recommended order

Rough ordering by effort-to-impact ratio, under the fixed memory ceiling:

1. Improvement 6 (hot-loop rewrite) — *free QPS, no tradeoff.*
2. Improvement 7 (SIMD distance + prefetch) — *free QPS.*
3. Improvement 1 (two-pass build + medoid start) — *free recall at target L.*
4. Improvement 8 (flat CSR layout) — *free RAM + free QPS.*
5. Improvement 2 (visited-set prune candidates) — *free recall.*
6. Improvement 4 (multi-entry + adaptive beam) — *improves tail latency.*
7. Improvement 5 (distance cascade / PQ + rerank) — *largest single lever under a tight memory ceiling.*
8. Improvement 10 (degree diagnostics + occlusion pruning) — *targeted graph-quality fix.*
9. Improvement 3 (adaptive α_i, R_i) — *works best on heterogeneous datasets.*
10. Improvement 9 (build-side contention fix) — *matters only if iteration speed becomes a bottleneck.*

12.2 Ablation protocol

For each stage, benchmark on fixed datasets and report:

- Recall@10, Recall@100.
- Mean and P99 latency; QPS.
- Mean distance computations per query.
- Build time and resident memory footprint at search time.
- Degree distribution and an average-hop-count proxy to the top-K basin.

Use one-change-at-a-time ablations, then cumulative composition to detect interactions. Pay special attention to interactions between Improvement 5 (PQ) and Improvement 8 (CSR): together they free substantial memory that can be reinvested in higher R, which is often the dominant recall lever.

13 Expected Outcomes

Under a fixed average degree budget and a fixed memory ceiling at search time, the combined effect of these ten modifications is expected to:

- Reduce required beam width L for a target recall (Improvements 1, 2, 3, 10).
- Increase raw QPS at fixed L (Improvements 6, 7, 8).
- Reduce high-percentile latency (Improvement 4).
- Free memory that can be reinvested into higher R or more candidates (Improvements 5, 8).
- Improve scaling of the build itself, enabling more experimentation (Improvement 9).

Conclusion

The roadmap preserves the spirit of Vamana while addressing the core limitations of the current baseline: construction path dependence, candidate impoverishment, parameter rigidity, entry-point brittleness, expensive exact distance overuse, inefficient search-loop data structures, unvectorized distance, scattered adjacency memory, build-side lock contention, and absence of degree-distribution feedback. Graph-quality upgrades (Improvements 1–3, 10), search-cost controls (Improvements 4–5), and implementation-level rewrites (Improvements 6–9) together give a principled path to better recall–latency–memory tradeoffs on the hidden evaluation dataset.